

DeveloperStore

AMBEV DEVELOPER EVALUATION

Technical and Architectural Decisions

Document consolidating every design decision, adopted pattern and technical rationale applied in the final solution of the senior-level challenge.

VERSION	DATE	RELEASE	COVERAGE	TESTS
2.0 — Final	May 2026	v1.0.0	93.3% Lines	284+ Automated

Table of Contents

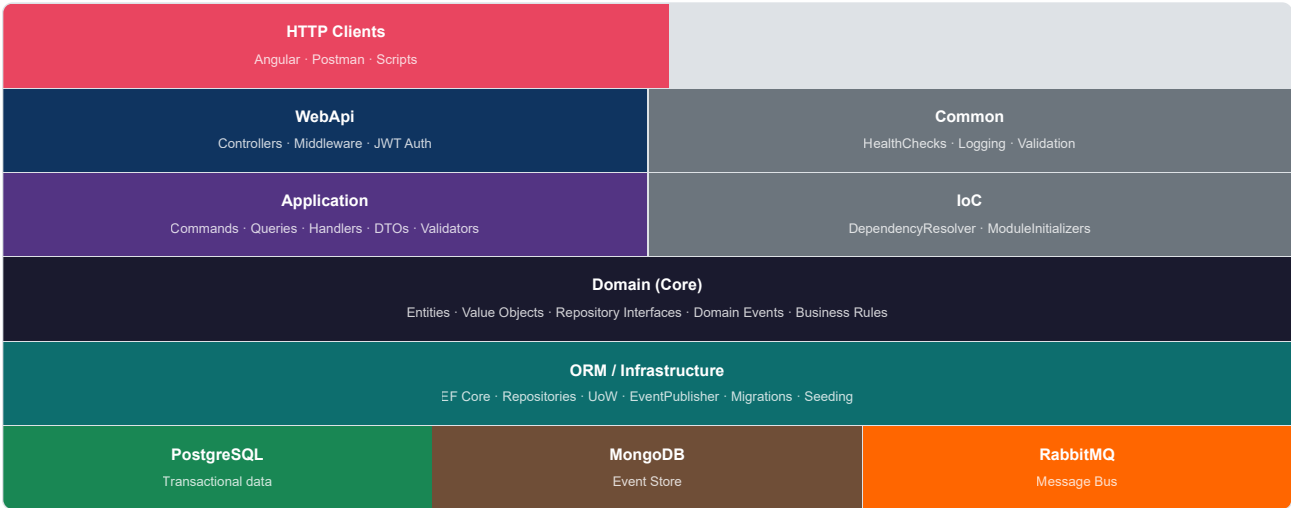
01	Architecture Overview
Clean Architecture · DDD · CQRS · Mediator	
02	Project Structure and Layers
Domain · Application · ORM · IoC · WebApi · Common	
03	Repository Pattern and Unit of Work
IRepository · IUnitOfWork · AsNoTracking · EF Core	
04	Domain Events System
IEventPublisher · Rebus · RabbitMQ · MongoDB · Decorator	
05	Authentication and Security
JWT HS256 · BCrypt · OWASP A01 · RBAC	
06	IoC Module and Dependency Initialization
DependencyResolver · IModuleInitializer · ModuleInitializers	
07	REST API Design
Conventions · Pagination · Filters · Validation · Responses	
08	Testing Strategy
Unit · Integration · Functional · Testcontainers · 93.3% Coverage	
09	GitFlow, Versioning and CI/CD
GitHub Actions · Docker · Conventional Commits · Release Tags	
10	Business Rules — Sales
Discounts · Cancellation · Domain Events	
—	Decision Summary Table
—	Frequently Asked Questions (FAQ)

Architecture Overview

Clean Architecture · Domain-Driven Design · CQRS · Mediator Pattern

The DeveloperStore solution is built on **Clean Architecture** combined with **Domain-Driven Design (DDD)** principles and the **CQRS** (Command Query Responsibility Segregation) pattern. This combination ensures a clear separation of concerns, maximum testability and low coupling between layers.

Clean Architecture Layers



The Dependency Rule

All dependencies point **inward**. The **Domain** knows about no other layer — it defines only interfaces (abstractions). ORM, Application and WebApi implement or consume these interfaces. IoC is the only project authorized to "see" multiple layers in order to register the concrete implementations.

CQRS via MediatR

Every operation is represented by a separate **Command** (write) or **Query** (read). MediatR automatically routes to the corresponding **Handler**. This eliminates fat controllers and centralizes the logic in the application handlers.



Complementary Patterns Applied

Pattern	Where It Is Applied	Main Benefit
Repository	Interfaces in Domain, implementations in ORM	Isolates persistence, eases testing
Unit of Work	IUnitOfWork + CommitAsync in every write handler	Guarantees transactional atomicity
Decorator	MongoEventPublisher → LoggingEventPublisher → RebusEventPublisher	Composed behaviors without inheritance
Pipeline Behavior	ValidationBehavior (MediatR)	Automatic validation before any handler
Module Initializer	IoC — ApplicationModuleInitializer, OrmModuleInitializer, WebApiModuleInitializer	Modular, testable initialization
Factory	IDesignTimeDbContextFactory in ORM	Migrations independent of WebApi

02

CODE ORGANIZATION

Project Structure and Layers

6 src projects + 3 test projects + feature-based organization

Production Projects (src/)

Project	Responsibility	Direct Dependencies
Domain	Entities, VOs, repository interfaces, domain events, pure business rules	<i>None</i> (innermost layer)
Application	Commands, Queries, Handlers, DTOs, Validators (FluentValidation), AutoMapper Profiles	Domain
ORM	DbContext, EF Core repositories, UnitOfWork, Migrations, Seeding, EventPublisher chain	Domain, Application
IoC	DependencyResolver, IModuleInitializer, per-module DI registrations	Domain, Application, ORM, WebApi, Common
WebApi	Controllers, Middleware, Program.cs, startup configuration	Application, IoC, Common
Common	Custom HealthChecks, Logging configuration (Serilog), security extensions, cross-cutting middlewares	Domain (optional)

Test Projects (tests/)

Project	Scope	Infrastructure	Tests
Unit	Handlers, Validators, Domain Entities	NSubstitute (pure mocks)	240
Integration	Repositories, DbContext, EF Queries	Testcontainers real PostgreSQL	41
Functional	End-to-end API, complete HTTP flows	Isolated Testcontainers + WebApplicationFactory	3

Feature-Based Organization (Vertical Slicing)

Inside `Application`, each aggregate has its own directory containing all the artifacts it needs:

```
Application/
├─ Sales/
│  ├─ CreateSale/ (CreateSaleCommand, CreateSaleResult, CreateSaleHandler, CreateSaleValidator)
│  ├─ GetSale/ (GetSaleQuery, GetSaleResult, GetSaleHandler)
│  ├─ UpdateSale/ ...
│  ├─ DeleteSale/ ...
│  └─ GetSales/ (pagination, filters, ordering)
├─ Users/ (same)
├─ Products/ (same)
└─ Carts/ (same)
```



Decision: Vertical Slicing per Aggregate

Keeping all artifacts of a use case in the same directory reduces navigation between files, isolates changes and eases onboarding of new developers.

Repository Pattern and Unit of Work

Abstractions in Domain · Implementations in ORM · AsNoTracking · CommitAsync

Interfaces in Domain (no EF dependency)

Every repository has its interface defined in the `Domain` project, ensuring the upper layers (Application) never depend on EF Core directly:

```
// Domain/Repositories/ISaleRepository.cs
public interface ISaleRepository
{
    Task<Sale> CreateAsync(Sale sale, CancellationToken ct = default);
    Task<Sale?> GetByIdAsync(Guid id, CancellationToken ct = default);
    Task<Sale> UpdateAsync(Sale sale, CancellationToken ct = default);
    Task<bool> DeleteAsync(Guid id, CancellationToken ct = default);
    IQueryable<Sale> GetAllQueryable();
}
```

IQueryable for Pagination and Filters

The `GetAllQueryable()` method returns `IQueryable<T>` instead of `IEnumerable<T>` or `List<T>`. This allows the *GetAll* handlers to apply filters, ordering and pagination directly in the database via the SQL generated by EF Core — avoiding loading unnecessary records into memory.

```
// ORM/Repositories/SaleRepository.cs
public IQueryable<Sale> GetAllQueryable()
=> _context.Sales.AsNoTracking()
    .Include(s => s.Items)
    .AsQueryable();
```



AsNoTracking on all reads

`AsNoTracking()` is applied on all read queries. EF Core does not need to track entities that will not be modified, resulting in lower memory usage and queries that are 20–40% faster on read-heavy workloads.

IUnitOfWork — Transactional Atomicity

The Unit of Work pattern ensures that multiple write operations are committed atomically. Every write handler calls `_unitOfWork.CommitAsync()` at the end and never calls `SaveChangesAsync()` directly on the repository.

```
// Domain/Repositories/IUnitOfWork.cs
public interface IUnitOfWork
{
    Task<int> CommitAsync(CancellationToken cancellationToken = default);
}

// ORM/Persistence/UnitOfWork.cs
public sealed class UnitOfWork : IUnitOfWork
{
    private readonly DefaultContext _context;
    public UnitOfWork(DefaultContext context) => _context = context;

    public Task<int> CommitAsync(CancellationToken ct = default)
    => _context.SaveChangesAsync(ct);
}
```

```
// Application/Sales/CreateSale/CreateSaleHandler.cs
public async Task<CreateSaleResult> Handle(CreateSaleCommand cmd, CancellationToken ct)
{
    var sale = _mapper.Map<Sale>(cmd);
    sale.CalculateTotals();

    await _saleRepository.CreateAsync(sale, ct);
    await _unitOfWork.CommitAsync(ct);    // ← single commit point

    await _eventPublisher.PublishAsync(new SaleCreatedEvent(sale.Id));
    return _mapper.Map<CreateSaleResult>(sale);
}
```

Implemented Repositories

Interface (Domain)	Implementation (ORM)	Entity
ISaleRepository	SaleRepository	Sale + SaleItem
IUserRepository	UserRepository	User
IProductRepository	ProductRepository	Product
ICartRepository	CartRepository	Cart + CartItem
IUnitOfWork	UnitOfWork	—

04

EVENTS AND MESSAGING

Domain Events System

IPublisher · Rebus/RabbitMQ · MongoDB Event Store · Decorator Pattern · Polly Retry

Contract in Domain

The `IPublisher` interface lives in the `Domain` project, ensuring the application handlers publish events without knowing the transport technology:

```
// Domain/Events/IPublisher.cs
public interface IPublisher
{
    Task PublishAsync<TEvent>(TEvent @event, CancellationToken ct = default)
        where TEvent : class;
}
```

Available Domain Events

Event	Raised in
<code>SaleCreatedEvent</code>	CreateSaleHandler — new sale created
<code>SaleModifiedEvent</code>	UpdateSaleHandler — sale updated
<code>SaleCancelledEvent</code>	DeleteSaleHandler — sale cancelled
<code>ItemCancelledEvent</code>	UpdateSaleHandler — item cancelled individually
<code>UserRegisteredEvent</code>	CreateUserHandler — new user registered

Decorator Chain: Three Composed Implementations

The Decorator pattern stacks three behaviors around each event publication, without inheritance and without changing the existing classes:

```
// Decorator chain (configured in IoC):
// MongoEventPublisher
//   └─ LoggingEventPublisher
//       └─ RebusEventPublisher ← final implementation (sends to RabbitMQ via Rebus)

public sealed class MongoEventPublisher : IEventPublisher
{
    private readonly IEventPublisher _inner; // LoggingEventPublisher
    private readonly IMongoCollection<DomainEventDocument> _collection;

    public async Task PublishAsync<TEvent>(TEvent @event, CancellationToken ct)
    {
        // 1. Persist to MongoDB (best-effort – does not block on failure)
        try { await _collection.InsertOneAsync(...); } catch { /* log only */ }

        // 2. Delegate to the next decorator
        await _inner.PublishAsync(@event, ct);
    }
}

public sealed class LoggingEventPublisher : IEventPublisher
{
    private readonly IEventPublisher _inner; // RebusEventPublisher
    private readonly ILogger _logger;

    public async Task PublishAsync<TEvent>(TEvent @event, CancellationToken ct)
    {
        _logger.LogInformation("Publishing {EventType}", typeof(TEvent).Name);
        await _inner.PublishAsync(@event, ct); // 3. Publish to the bus via Rebus
    }
}

public sealed class RebusEventPublisher : IEventPublisher
{
    private readonly IBus _bus;
    public async Task PublishAsync<TEvent>(TEvent @event, CancellationToken ct)
    => await _bus.Publish(@event);
}
```

Rebus + RabbitMQ

Rebus is the messaging framework that provides an abstraction over the transport. In production it uses **RabbitMQ 3.13**; in development and tests it uses the **InMemory** transport, removing the need for a running RabbitMQ in the unit tests.

```
// IoC – conditional transport configuration
if (env.IsDevelopment() || env.IsEnvironment("Testing"))
    rebus.Transport(t => t.UseInMemoryTransport(network, "developer-store"));
else
    rebus.Transport(t => t.UseRabbitMq(connectionString, "developer-store"));
```

Four Message Handlers

Each event type has a dedicated `IHandleMessages<T>` that processes the message asynchronously:

Handler	Consumed Event	Action
<code>SaleCreatedEventHandler</code>	<code>SaleCreatedEvent</code>	Log + audit
<code>SaleModifiedEventHandler</code>	<code>SaleModifiedEvent</code>	Log + audit
<code>SaleCancelledEventHandler</code>	<code>SaleCancelledEvent</code>	Log + audit
<code>ItemCancelledEventHandler</code>	<code>ItemCancelledEvent</code>	Log + audit

MongoDB Event Store

All domain events are persisted to MongoDB as a **DomainEventDocument** before being sent to the bus. The operation is *best-effort*: a MongoDB failure does not prevent publication to RabbitMQ. This creates an immutable audit trail of the domain events:


```
public record DomainEventDocument
{
    public ObjectId Id { get; init; }
    public string EventType { get; init; }
    public string Payload { get; init; } // serialized JSON
    public DateTime OccurredAt { get; init; }
}
```

Retry with Polly

Event publication has a retry policy configured with **Polly**: 3 attempts with exponential backoff (1s → 2s → 4s) on a transient network or broker failure:

```
Policy
    .Handle<Exception>()
    .WaitAndRetryAsync(3, attempt => TimeSpan.FromSeconds(Math.Pow(2, attempt - 1)))
```

05

IDENTITY AND ACCESS CONTROL

Authentication and Security

JWT HS256 · BCrypt · OWASP A01 · Role-Based Authorization

JWT HS256 with Strict Validation

Authentication uses **JWT** with the **HS256** (HMAC-SHA256) algorithm. The secret key is validated at application startup — keys shorter than 32 bytes throw an exception and prevent startup:

```
// Startup validation
if (jwtKey.Length < 32)
    throw new InvalidOperationException(
        "JWT secret key must be at least 32 characters for HS256 security.");
```

Tokens include validation of **Issuer**, **Audience**, **Expiration** and **Signature**. Any invalid token results in HTTP 401 before reaching the controller.

BCrypt for Passwords

Passwords are never stored in plain text. The hash is generated with **BCrypt** (work factor 12), which includes an automatic salt and is resistant to brute-force and rainbow-table attacks:

```
// Hash when creating a user
user.Password = BCrypt.Net.BCrypt.HashPassword(command.Password);

// Verification on login
BCrypt.Net.BCrypt.Verify(command.Password, user.Password)
```

OWASP A01 — Ownership Check

The `PUT /api/users/{id}` endpoint implements object-level access control, addressing one of the most critical vulnerabilities in the **OWASP Top 10 (A01 — Broken Access Control)**. A user can only update their own profile:

```
// UsersController.cs
[HttpPut("{id}")]
[Authorize]
public async Task<IActionResult> UpdateUser(Guid id, UpdateUserRequest request)
{
    var authenticatedUserId = Guid.Parse(
        User.FindFirstValue(ClaimTypes.NameIdentifier!));

    if (authenticatedUserId != id)
        return Forbid(); // HTTP 403 – not the resource owner

    // ...
}
```

Role-Based Authorization (RBAC)

The user role-change endpoint requires the **Admin** role:

```
// Only administrators can change roles
[HttpPatch("{id}/role")]
[Authorize(Roles = "Admin")]
public async Task<IActionResult> UpdateUserRole(Guid id, UpdateUserRoleRequest request)
{
    // ...
}
```

Security Summary Table

Aspect	Implementation	Standard/Reference
Authentication	JWT Bearer HS256	RFC 7519
Password hash	BCrypt work factor 12	OWASP Password Storage
Ownership check	ClaimTypes.NameIdentifier vs route id	OWASP A01:2021
Role check	[Authorize(Roles = "Admin")]	RBAC
Key validation	Minimum 32 bytes at startup	NIST SP 800-107
Public endpoints	POST /api/auth/login, POST /api/users	–

IoC Module and Dependency Initialization

DependencyResolver · IModuleInitializer · Three ModuleInitializers

Instead of registering all dependencies directly in `Program.cs`, the application uses a dedicated **IoC** project with a module pattern, keeping startup clean and the modules independently testable.

IModuleInitializer — Module Interface

```
// IoC/IModuleInitializer.cs
public interface IModuleInitializer
{
    void Initialize(IServiceCollection services, IConfiguration configuration);
}
```

Three ModuleInitializers

Module	Responsibility
<code>ApplicationModuleInitializer</code>	Registers MediatR handlers, AutoMapper profiles, FluentValidation validators, the ValidationBehavior pipeline
<code>OrmModuleInitializer</code>	Configures EF Core + PostgreSQL, registers repositories, UnitOfWork, the EventPublisher chain (Decorator), Rebus/RabbitMQ, MongoDB client
<code>WebApiModuleInitializer</code>	Configures JWT Authentication, role-based authorization, Swagger/OpenAPI, HealthChecks, CORS, Serilog

DependencyResolver — Entry Point

```
// IoC/DependencyResolver.cs
public static class DependencyResolver
{
    public static IServiceCollection AddDeveloperStore(
        this IServiceCollection services,
        IConfiguration config,
        IHostEnvironment env)
    {
        var initializers = new IModuleInitializer[]
        {
            new ApplicationModuleInitializer(),
            new OrmModuleInitializer(),
            new WebApiModuleInitializer()
        };

        foreach (var initializer in initializers)
            initializer.Initialize(services, config);

        return services;
    }
}
```

```
// WebApi/Program.cs — extremely clean startup
builder.Services.AddDeveloperStore(builder.Configuration, builder.Environment);
```

★ Benefit: a one-line Program.cs

The IoC project absorbs all the configuration complexity. `Program.cs` is left with a single extension call, easing maintenance, onboarding and integration tests that need to swap modules.

REST API Design

Conventions · Pagination · Filters and Ordering · Validation · Standardized Responses

Available Endpoints

Resource	Endpoints	Auth Required
Auth	POST /api/auth/login	No
Users	POST · GET · PUT /api/users/{id} · PATCH /api/users/{id}/role · DELETE	Partial (POST open)
Products	POST · GET · GET/{id} · PUT · DELETE /api/products	Yes
Carts	POST · GET · GET/{id} · PUT · DELETE /api/carts	Yes
Sales	POST · GET · GET/{id} · PUT · DELETE /api/sales	Yes

Standardized Pagination

All listing endpoints follow the same pagination contract via query params:

```
GET /api/sales?page=1&size=10&order=date+desc

{
  "data": [...],
  "totalCount": 47,
  "totalPages": 5,
  "currentPage": 1
}
```

Filters and Ordering

The GET (listing) endpoints support server-side filtering. Examples of available filters:

Endpoint	Available Filters
GET /api/sales	customerId , branchId , minDate , maxDate , minTotal , isCancelled , order
GET /api/carts	userId , minDate , maxDate , order
GET /api/products	title , minPrice , maxPrice , category , order
GET /api/users	username , email , role , order

Automatic Validation via ValidationBehavior

The MediatR pipeline includes a `ValidationBehavior<TRequest, TResponse>` that runs the corresponding FluentValidation validator before any handler. On a validation failure, MediatR returns a `ValidationException` that the global middleware converts into HTTP 400 with the error details:

```
{
  "type": "ValidationError",
  "errors": {
    "customerId": ["'Customer Id' must not be empty."],
    "items": ["Items list must not be empty."]
  }
}
```

Standardized HTTP Responses

Situation	HTTP Status	Format
Successful creation	201 Created	<code>{ data: {...} }</code>
Successful read	200 OK	<code>{ data: {...} }</code> or paginated
Validation failed	400 Bad Request	<code>{ type, errors: {...} }</code>
Not authenticated	401 Unauthorized	—
No permission	403 Forbidden	—
Resource not found	404 Not Found	<code>{ error: "... " }</code>

08

QUALITY AND RELIABILITY

Testing Strategy

93.3% Coverage · 284+ Automated Tests · Testcontainers · Three Layers

93.3%

LINE COVERAGE

93.6%

BRANCH COVERAGE

240

UNIT TESTS

44

INTEGRATION + FUNC. TESTS

Layer 1 — Unit Tests (240)

They test handlers, validators and domain entities in complete isolation. They use **NSubstitute** to replace dependencies. They need no database or infrastructure.

Domain	Test Classes	Cases
Sales	CreateSaleHandler, GetSaleHandler, UpdateSaleHandler, DeleteSaleHandler, GetSalesHandler + Validators + Entity	~80
Users	CreateUserHandler, GetUserHandler, UpdateUserHandler, DeleteUserHandler + Validators	~55
Products	CreateProductHandler, GetProductHandler, UpdateProductHandler, DeleteProductHandler + Validators	~55
Carts	CreateCartHandler (3), GetCartHandler (2), UpdateCartHandler (3), DeleteCartHandler (2), GetCartsHandler (7)	17
Auth	CreateTokenHandler, GetTokenHandler	~10
Others	Domain entities, value objects, business rules	~23

Layer 2 — Integration Tests (41)

They test EF Core repositories against a **real PostgreSQL** provisioned by **Testcontainers**. They verify that the generated SQL queries, migrations and mapping configurations are correct.

```
// Shared fixture - a single PostgreSQL container for the whole integration suite
[Collection("Integration")]
public class SaleRepositoryTests : IClassFixture<DatabaseFixture>
{
    public SaleRepositoryTests(DatabaseFixture fixture) { ... }
}
```

✓ Testcontainers with ICollectionFixture

Using **ICollectionFixture** ensures that only **one PostgreSQL container** starts for the entire integration test collection, drastically reducing execution time (from ~120s to ~15s for 41 tests).

Layer 3 — Functional Tests (3)

End-to-end tests via ASP.NET Core's **WebApplicationFactory**, testing complete HTTP flows (request → middleware → handler → database → response). Each functional test uses its own isolated Testcontainers container.

Coverage: Details

Metric	Covered	Total	Percentage
Lines	1,367	1,465	93.31%
Branches	176	188	93.61%

Coverage scope: Domain and Application projects only — the layers with business logic. ORM and WebApi are excluded as they are heavily covered by the integration and functional tests.

Running the Tests

```
# Unit tests with coverage
dotnet test --collect:"XPlat Code Coverage" --filter "FullyQualifiedName~Unit"

# Integration tests (requires Docker Desktop running)
dotnet test --filter "FullyQualifiedName~Integration"

# Functional tests (requires Docker Desktop running)
dotnet test --filter "FullyQualifiedName~Functional"

# HTML coverage report
./coverage-report.ps1
```


GitFlow, Versioning and CI/CD

GitHub Actions · Docker · Conventional Commits · SemVer Tags

Strict GitFlow

The project follows **GitFlow** strictly. There are never direct commits to `main`. Every feature goes through a dedicated branch and a `--no-ff` merge:

Branch	Purpose	Merge target
<code>main</code>	Production code — releases only	—
<code>develop</code>	Integration branch — always stable	main (via release)
<code>feature/*</code>	New features	develop
<code>fix/*</code>	Bug fixes	develop
<code>docs/*</code>	Documentation updates	develop
<code>refactor/*</code>	Refactorings with no behavior change	develop
<code>chore/*</code>	Maintenance (deps, CI, config)	develop

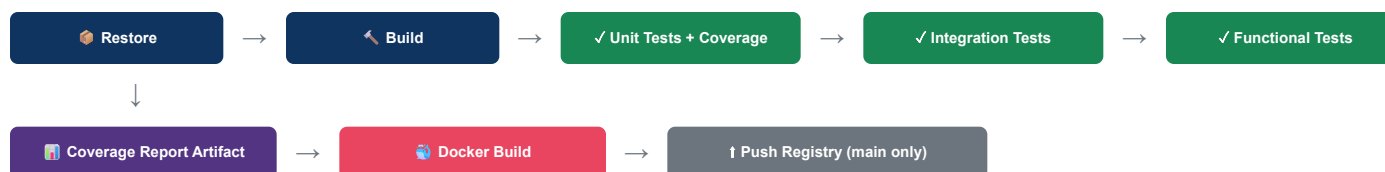
Conventional Commits

Every commit follows the **Conventional Commits** specification, enabling automatic changelog generation and semantic versioning:

```
feat: add discount calculation for bulk sales
fix: correct total amount rounding in cart items
test: add cart handler unit tests (18 cases)
refactor: extract IUnitOfWork to domain project
docs: update README with final coverage metrics
chore: configure GitHub Actions CI workflow
```

CI/CD Pipeline — GitHub Actions

The `.github/workflows/ci.yml` file defines a pipeline with two jobs:



Job: build-and-test

```
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with: { dotnet-version: '8.0.x' }
      - run: dotnet restore
      - run: dotnet build --no-restore
      - run: dotnet test --filter Unit --collect:"XPlat Code Coverage"
      - run: dotnet test --filter Integration # Testcontainers provision automatically
      - run: dotnet test --filter Functional
      - run: dotnet tool install -g dotnet-reportgenerator-globaltool
      - run: reportgenerator -reports:**/coverage.cobertura.xml -targetdir:coverage-html
      - uses: actions/upload-artifact@v4
        with: { name: coverage-report, path: coverage-html }
```

Job: docker (needs: build-and-test)

```
docker:
  needs: build-and-test
  steps:
    - run: docker build -t developer-store .
    - if: github.ref == 'refs/heads/main'
      run: docker push ${ env.REGISTRY }}/developer-store
```

SemVer Versioning and GitHub Releases

The project uses annotated tags with **Semantic Versioning**. The current version is `v1.0.0`, available as a GitHub Release with automatic changelogs.

```
git tag -a v1.0.0 -m "chore: release v1.0.0 - full implementation"
git push origin v1.0.0
gh release create v1.0.0 --title "v1.0.0 - DeveloperStore Final" --generate-notes
```

Business Rules — Sales

Quantity-based discounts · Cancellation · Domain events · Consistency

Quantity-Based Discount Calculation

The `Sale` entity implements automatic discount rules applied item by item at creation and update time:

Item quantity	Discount applied
1 to 3 units	0% — no discount
4 to 9 units	10% on the item subtotal
10 to 20 units	20% on the item subtotal
More than 20 units	Forbidden — <code>ValidationException</code> thrown

```
// Domain/Entities/SaleItem.cs
public void ApplyDiscount()
{
    Discount = Quantity switch
    {
        >= 10 and <= 20 => 0.20m,
        >= 4           => 0.10m,
        _              => 0.00m
    };

    if (Quantity > 20)
        throw new DomainException("Cannot sell more than 20 identical items.");

    TotalAmount = UnitPrice * Quantity * (1 - Discount);
}
```

Sale and Item Cancellation

Sales and individual items can be cancelled. Cancellation is represented via domain events, ensuring full traceability:

```
// Cancel the whole sale → SaleCancelledEvent
// Cancel a specific item → ItemCancelledEvent (item stays in the list, IsCancelled = true)
sale.CancelItem(itemId); // validates existence, sets IsCancelled, recalculates the total
```

Total Consistency

The `CalculateTotals()` method recalculates the `TotalAmount` of the whole sale by summing only non-cancelled items. It is called on every write operation before the commit:

```
public void CalculateTotals()
{
    foreach (var item in Items.Where(i => !i.IsCancelled))
        item.ApplyDiscount();

    TotalAmount = Items
        .Where(i => !i.IsCancelled)
        .Sum(i => i.TotalAmount);
}
```

Events Raised per Operation

Operation	Event(s) Raised
POST /api/sales	SaleCreatedEvent
PUT /api/sales/{id}	SaleModifiedEvent + ItemCancelledEvent (per cancelled item)
DELETE /api/sales/{id}	SaleCancelledEvent
POST /api/users	UserRegisteredEvent

Technical Decision Summary Table

Every architectural choice consolidated with rationale and considered alternatives

Decision	Adopted Choice	Considered Alternative	Rationale
Architecture	Clean Architecture + DDD	Simple MVC	Separation of concerns, testability, independent evolution of layers
Command Pattern	CQRS via MediatR	Service Layer	Decouples reads from writes, enables a behavior pipeline, cohesive handlers
Data Access	Repository Pattern + EF Core	Dapper / direct ORM	Testable abstraction, automatic migrations, type-safe LINQ
Unit of Work	IUnitOfWork + CommitAsync	SaveChanges in the repository	Single commit point, explicit transaction control in the handler
Messaging	Rebus + RabbitMQ	MediatR Notifications / NServiceBus	Pluggable transport (InMemory in tests), code decoupled from the broker
Event Store	MongoDB (DomainEventDocument)	Log only / PostgreSQL Event Table	Flexible schema for heterogeneous events, immutable audit trail
Event Composition	Decorator (Mongo → Logging → Rebus)	Single class with multiple responsibilities	SRP: each decorator has a single responsibility; composition without inheritance
Authentication	JWT HS256 + BCrypt	Session / external OAuth	Stateless, horizontally scalable, secure with a key ≥ 32 bytes
IoC / DI	IoC module with IModuleInitializer	Direct registrations in Program.cs	Clean Program.cs, independent modules, eases startup tests
Integration Tests	Testcontainers + real PostgreSQL	SQLite in-memory	Avoids divergence between SQLite and production PostgreSQL behavior
Container Sharing	ICollectionFixture (single container)	One container per test class	Reduces CI time from ~120s to ~15s for 41 integration tests
Event Resilience	Polly exponential retry (3×)	No retry / Circuit Breaker	Handles transient network/broker failures without losing the event
Migrations	IDesignTimeDbContextFactory in ORM	Migrations in WebApi	ORM owns the schema; WebApi need not reference EF Core directly
CI/CD	GitHub Actions (multi-job)	Jenkins / GitLab CI	Native to the GitHub repository, zero extra infrastructure, conditional Docker push
Versioning	SemVer + Conventional Commits	Date-based versions	Automatic changelog, clarity on the nature of a change (breaking vs minor vs patch)

Frequently Asked Questions (FAQ)

Answers to the main questions about the project's choices

? Why Rebus instead of using RabbitMQ.Client directly?

Rebus provides a transport abstraction — the same publishing code works with `InMemoryTransport` (tests), RabbitMQ (production) or any other supported backend. Switching brokers requires only a configuration change, not a code change.

? Why MongoDB for events and not a PostgreSQL table?

Domain events are heterogeneous documents — each type has a different payload. MongoDB accommodates this naturally with a flexible schema. In addition, MongoDB is an explicit requirement of the challenge as a second database.

? Is Redis required?

No. Redis was never a requirement of the challenge specification. The docker-compose provisions only PostgreSQL, MongoDB and RabbitMQ. The README was updated to remove any earlier, incorrect mention of Redis.

? Why Testcontainers instead of a dedicated test database?

Testcontainers removes environment dependencies — any machine with Docker can run the tests. The PostgreSQL database starts on demand and is destroyed after the tests, leaving no residual state and requiring no manual setup.

? Why `ICollectionFixture` instead of `IClassFixture` ?

With `IClassFixture`, each test class starts and destroys its own container — 41 start/stop cycles. With `ICollectionFixture`, a single container serves the entire integration test collection, drastically reducing the total time.

? Why the Decorator instead of a single EventPublisher with multiple responsibilities?

SRP (Single Responsibility Principle). `MongoEventPublisher` only persists to MongoDB; `LoggingEventPublisher` only logs; `RebusEventPublisher` only publishes to the bus. Adding or removing a behavior (e.g. adding an event cache) requires creating a decorator, not modifying existing code (OCP).

? How does the OWASP A01 ownership check work?

The JWT's `NameIdentifier` claim is extracted by the authentication middleware and compared with the route id (`{id}`). If they differ, it returns 403 Forbidden before running any business logic. This prevents user A from changing user B's data even with a valid token.

? Does the 93% coverage include all projects?

No — intentionally. The scope covers **Domain** and **Application**, which concentrate 100% of the business logic. ORM (EF Core repositories) and WebApi (controllers) are covered by the integration and functional tests, which run against a real database.

? How do I reproduce the full environment locally?

```
git clone https://github.com/raphaelfernandesribeiro/desafio-tecnico
cd desafio-tecnico/template/backend
docker-compose up -d          # PostgreSQL + MongoDB + RabbitMQ
dotnet run --project src/Ambev.DeveloperEvaluation.WebApi
```